CirculAR — Arquitectura backend v1

1. Objetivo

Definir la arquitectura backend inicial de CirculAR para implementar el producto sobre una base robusta, modular, trazable y expansible.

Este documento fija:

- stack tecnológico recomendado
- criterios de arquitectura y modularidad
- estructura real del backend
- capas y responsabilidades
- estrategia de persistencia
- lineamientos de asincronía
- storage de imágenes
- concurrencia e idempotencia
- observabilidad
- testing
- seguridad
- roadmap técnico de arranque

Se apoya en:

- CirculAR — Blueprint inicial
- CirculAR — Definiciones oficiales v2
- CirculAR — Dominio oficial v1
- CirculAR — Estados e invariantes v1
- CirculAR — Contratos API v1

---

2. Principios arquitectónicos

2.1 Backend como fuente de verdad

El backend define:

- estados oficiales
- transiciones válidas
- ownership
- restricciones operativas
- acciones disponibles

El frontend consume surfaces claras y no infiere lógica de negocio crítica.

2.2 Modularidad por bounded context

La arquitectura se organiza por dominios funcionales con límites claros. Cada módulo debe tener:

- responsabilidad concreta
- contratos explícitos
- bajo acoplamiento
- posibilidad de evolucionar sin romper el resto del sistema

2.3 Separación entre escritura y lectura

La base operativa del sistema debe distinguir:

- comandos y mutaciones con reglas de dominio
- read surfaces pensadas para UI

No conviene exponer entidades ORM crudas al frontend.

2.4 Estados explícitos y transiciones concentradas

Los cambios de estado relevantes deben resolverse en servicios de aplicación o domain services bien definidos. No deben quedar dispersos en controllers, repositories o frontend.

2.5 Asincronía preparada desde el inicio

Aunque el MVP sea simple, el backend debe quedar listo para:

- auditoría de imágenes
- expiración de reservas
- cierre automático de conversaciones
- notificaciones en tiempo real
- recomputación de reputación
- tareas batch o reintentos

2.6 Trazabilidad obligatoria

Toda decisión importante debe poder auditarse:

- quién ejecutó una acción
- sobre qué recurso
- desde qué estado
- hacia qué estado
- cuándo ocurrió
- por qué ocurrió

---

3. Stack recomendado v1

3.1 Lenguaje y framework

- **TypeScript**
- **NestJS** como framework principal

Motivos:

- modularidad natural por dominios
- buen encaje con DTOs, validación y guards
- ecosistema sólido para auth, websockets, jobs y testing
- buena mantenibilidad para backend enterprise-like

3.2 Base de datos

- **PostgreSQL**

Motivos:

- consistencia transaccional
- buen soporte para constraints e índices parciales
- JSONB cuando haga falta flexibilidad controlada
- madurez para modelado relacional fuerte

3.3 ORM

Recomendación principal para este proyecto:

- **TypeORM**

Motivo principal:

- encaja bien con NestJS
- migraciones conocidas
- suficiente para MVP robusto
- permite control razonable del modelo sin empujar a sobreabstracción innecesaria

Condición:

- no usar entidades ORM como contrato público
- mantener repositories y mappers claros

3.4 API contract

- **REST JSON versionado desde `/v1`**
- **OpenAPI / Swagger**

3.5 Auth

- **JWT access token + refresh token**
- refresh token persistido con control de revocación
- password hashing con **Argon2**

3.6 Storage de archivos

- **S3-compatible object storage**
- CDN por delante cuando el volumen lo justifique

3.7 Realtime

- **WebSockets con NestJS Gateway** para notificaciones y eventos de conversación en tiempo real

3.8 Procesos asincrónicos

- **Queue basada en Redis**, por ejemplo BullMQ

3.9 Validación

- `class-validator`
- `class-transformer`
- validaciones adicionales de dominio fuera del DTO cuando corresponda

3.10 Testing

- **Vitest** o **Jest** para unit/integration
- e2e con **Supertest**

---

4. Arquitectura lógica general

La arquitectura recomendada es una variante pragmática de arquitectura modular por dominio con layering interno.

4.1 Estructura general

Cada bounded context vive como módulo de negocio. Dentro de cada módulo conviene separar:

- **domain**
- **application**
- **infrastructure**
- **presentation**

- **read-models** cuando aplique

4.2 Capas y responsabilidades

Domain

Contiene el núcleo del negocio:

- entidades de dominio
- value objects
- enums oficiales
- invariantes
- eventos de dominio
- interfaces de repositorio si querés mantener pureza de dominio
- domain services

No debería depender de NestJS ni del ORM.

Application

Orquesta casos de uso:

- comandos
- queries
- application services
- policies de autorización
- coordinación entre repositorios
- transacciones
- publicación de eventos

Es donde deben vivir operaciones como:

- aceptar interacción
- reservar listing
- cerrar incompatibles
- abrir match session
- confirmar éxito
- renovar reserva

Infrastructure

Implementa detalles técnicos:

- repositories TypeORM
- entidades ORM
- adaptadores externos
- storage provider
- queue producers/consumers
- websocket emitters
- notifiers

Presentation

Expone interfaces de entrada:

- controllers REST
- gateways websocket
- DTOs request/response
- pipes / guards / interceptors

Read models

Pensados para UI backend-driven:

- listing detail view
- feed card view
- incoming interactions view
- match coordination view
- bootstrap surface futura

---

5. Módulos oficiales del backend

Se recomienda esta estructura de alto nivel:

- `auth`
- `users`
- `profiles`
- `trust`
- `catalog`
- `listings`
- `discovery`
- `saved-listings`
- `interactions`
- `matches`
- `conversations`
- `moderation`
- `reputation`
- `notifications`
- `common`
- `config`
- `health`

5.1 Criterio de corte

No todos los bounded contexts necesitan un micro-módulo independiente desde el día 1. El objetivo no es fragmentar de más, sino mantener ownership claro.

Para MVP, una división sana sería:

- `auth`
- `profiles`
- `listings`
- `discovery`
- `interactions`
- `matches`
- `moderation`
- `reputation`
- `notifications`
- `common`

Con `users`, `trust`, `catalog` y `conversations` como submódulos internos o subdominios según convenga.

---

6. Ownership técnico por módulo

6.1 Auth

Responsable de:

- registro
- login
- refresh
- logout
- emisión y revocación de tokens
- identidad del usuario autenticado

No debe conocer lógica comercial de listings o interacciones.

6.2 Profiles

n
Responsable de:

- perfil editable
- perfil público
- reach zones
- señales visibles básicas de confianza no derivadas

6.3 Listings

Responsable de:

- garment data
- listing lifecycle
- commercial config
- fotos asociadas
- estado oficial de disponibilidad
- submit a review
- pausa / archivo / renovación de reserva

6.4 Discovery

Responsable de:

- feed swipe
- filtros
- guardados
- descarte
- selección de read models para exploración

No debe mutar estados comerciales nucleares del listing.

6.5 Interactions

Responsable de:

- purchase intents
- trade proposals
- aceptación / rechazo / cancelación
- cierre automático de incompatibles

6.6 Matches

Responsable de:

- match session
- coordinación post-aceptación
- confirmación de éxito
- fallo / cancelación / expiración

6.7 Conversations

Responsable de:

- thread contextual
- mensajes
- quick actions

- restricciones post-cierre

Puede vivir dentro de `matches` en MVP para evitar sobrefragmentación.

6.8 Moderation

Responsable de:

- auditoría de imágenes
- moderación estructurada
- reasons tipadas
- observed / rejected / approved
- escalamiento posterior

6.9 Reputation

Responsable de:

- reputation entries
- métricas agregadas
- success rate
- response time agregado
- cancelaciones derivadas

6.10 Notifications

Responsable de:

- persistencia de notificaciones
- entrega realtime
- read/unread
- colas de eventos visibles al usuario

---

7. Estructura de carpetas recomendada

```
src/
  main.ts
  app.module.ts

  config/
    env/
    app.config.ts
    database.config.ts
    storage.config.ts
    queue.config.ts
    swagger.config.ts
```

```
common/
  domain/
    enums/
    types/
    constants/
  application/
    base/
    ports/
  infrastructure/
    db/
    events/
    queue/
    logging/
  presentation/
    dto/
    guards/
    interceptors/
    filters/
    decorators/

modules/
  auth/
    domain/
    application/
    infrastructure/
    presentation/
  profiles/
    domain/
    application/
    infrastructure/
    presentation/
  listings/
    domain/
    application/
    infrastructure/
    presentation/
    read-models/
  discovery/
    application/
    infrastructure/
    presentation/
    read-models/
  interactions/
    domain/
    application/
    infrastructure/
    presentation/
```

```
    read-models/
  matches/
    domain/
    application/
    infrastructure/
    presentation/
    read-models/
  moderation/
    domain/
    application/
    infrastructure/
    presentation/
  reputation/
    domain/
    application/
    infrastructure/
    presentation/
  notifications/
    domain/
    application/
    infrastructure/
    presentation/

shared/
  kernel/
  value-objects/
  errors/
  result/
```

---

8. Diseño de persistencia

8.1 Estrategia general

Persistencia relacional fuerte en PostgreSQL para el núcleo transaccional.

Principio:

- los agregados principales deben mapearse a tablas claras
- constraints de base deben reforzar invariantes importantes
- no delegar toda la consistencia al código de aplicación

8.2 Entidades transaccionales esperadas

Base mínima sugerida:

- `users`
- `sessions`
- `public_profiles`
- `trust_profiles`
- `reach_zones`
- `reputation_profiles`
- `listings`
- `garments`
- `listing_photos`
- `listing_trade_preferences`
- `saved_listings`
- `purchase_intents`
- `trade_proposals`
- `trade_proposal_items`
- `match_sessions`
- `conversation_threads`
- `conversation_messages`
- `moderation_reviews`
- `image_audits`
- `reputation_entries`
- `notifications`
- `user_reports`
- `domain_events_outbox`

8.3 Read models

No hace falta materializar todo desde el día 1.

Recomendación MVP:

- construir read surfaces con queries optimizadas y DTO assemblers
- materializar solo cuando haya dolor real de performance o complejidad

Posibles proyecciones futuras:

- feed candidate table
- profile trust snapshot
- interactions inbox projection
- match summary projection

8.4 Soft delete

Aplicar soft delete en recursos donde tenga sentido operativo:

- listings archivados

- conversation archival técnico
- notificaciones antiguas si se define política

No usar soft delete indiscriminado en todo. En algunos casos conviene estado explícito más `archivedAt`.

---

9. Estrategia de transacciones

9.1 Regla general

Toda operación que cambie estado entre agregados relacionados debe ejecutarse transaccionalmente.

Casos críticos:

- aceptar PurchaseIntent
- aceptar TradeProposal
- reservar listing
- cerrar interacciones incompatibles
- abrir MatchSession
- crear ConversationThread
- confirmar éxito y cerrar listing

9.2 Unidad transaccional recomendada

Una aceptación debe ser una sola transacción lógica.

Ejemplo:

1. validar ownership y estado
2. verificar que listing siga PUBLISHED
3. marcar interacción aceptada
4. pasar listing a RESERVED
5. expirar/rechazar incompatibles
6. crear match session
7. crear conversation thread
8. registrar outbox events

Todo eso debe quedar atómico.

9.3 Outbox pattern

Para evitar inconsistencias entre DB y procesos externos, conviene implementar desde el inicio un **outbox pattern simple**.

Cuando una operación relevante se confirma:

- se persiste el cambio de negocio
- se persiste el evento en `domain_events_outbox`
- un worker posterior publica notificaciones o dispara jobs externos

Esto evita perder eventos si la app cae entre commit y publish.

---

10. Concurrencia e idempotencia

Este es uno de los puntos más importantes del sistema.

10.1 Riesgos reales

- dos usuarios intentando aceptar al mismo tiempo
- dos propuestas compitiendo por el mismo listing
- un proposed item comprometido en múltiples operaciones incompatibles
- doble submit por reintentos del cliente
- workers repitiendo jobs

10.2 Estrategia recomendada

Combinar:

- transacciones SQL
- row-level locking en operaciones críticas
- índices únicos parciales cuando corresponda
- idempotency keys para mutaciones sensibles

10.3 Locks críticos

Usar `SELECT ... FOR UPDATE` o equivalente sobre:

- fila de `listings` al aceptar una interacción
- fila de `purchase_intents` o `trade_proposals` a resolver
- filas de proposed items afectados cuando haya riesgo de compromiso simultáneo

10.4 Restricciones recomendadas

Ejemplos conceptuales:

- una sola interacción activa por usuario y listing en cada tipo
- una sola match session no terminal por listing
- proposed item no comprometido en más de una sesión incompatible

10.5 Idempotencia

Mutaciones sensibles deberían aceptar `Idempotency-Key` en header, al menos para:

- register opcionalmente
- create listing
- submit review
- create purchase intent
- create trade proposal
- accept / reject interaction
- confirm success

Esto reduce duplicados por retry móvil o mala conectividad.

---

11. Storage e imágenes

11.1 Principio general

Las imágenes no deben persistirse en la base como binario. Solo metadatos y referencias.

11.2 Flujo recomendado

1. cliente solicita upload o sube vía endpoint backend controlado
2. archivo se guarda en bucket temporal o definitivo según estrategia
3. backend registra `listing_photo`
4. foto entra en pipeline de auditoría
5. resultado actualiza `image_audits` y eventualmente `moderation_reviews`

11.3 Modelo de archivo sugerido

Guardar:

- object key
- public URL o CDN URL
- mime type
- width/height
- size bytes
- checksum/hash
- uploadedAt
- audit status
- position

11.4 Estrategia inicial simple

Para MVP robusto sin sobrecomplejidad:

- upload al backend

- backend valida tamaño / formato / cantidad
- backend sube a storage
- backend dispara job de auditoría

Más adelante se puede migrar a presigned URLs si el volumen lo justifica.

---

12. Pipeline de moderación e IA

12.1 Diseño recomendado

La moderación no debe bloquear con lógica improvisada dentro del controller.

Debe resolverse como pipeline desacoplado:

- validaciones sin IA
- auditoría barata
- auditoría IA cuando haga falta
- decisión estructurada

12.2 Etapas sugeridas

Etapa 1: validaciones baratas

- cantidad mínima/máxima de fotos
- formato permitido
- tamaño permitido
- resolución mínima
- duplicados obvios

Etapa 2: auditoría automática barata

- blur
- exposición / iluminación
- detección básica de objeto visible

Etapa 3: validación IA aplicada

- contenido inapropiado
- no-prenda
- mismatch básico con categoría

Etapa 4: decisión de moderación

Resultado estructurado:

- APPROVED
- OBSERVED

- REJECTED

12.3 Regla operacional

Un listing no pasa a `PUBLISHED` sin review vigente aprobada.

12.4 Adaptador de proveedor

Nunca acoplar el dominio a un proveedor específico. Crear un puerto tipo:

- `ImageModerationProvider`

Con response estructurada y mapeo interno.

---

13. Notificaciones y realtime

13.1 Tipos de notificación MVP

- interés recibido
- propuesta recibida
- interacción aceptada
- interacción rechazada
- nuevo mensaje en conversación
- reserva próxima a expirar o expirada
- publicación observada
- publicación rechazada

13.2 Estrategia técnica

Persistir siempre la notificación. Realtime es una capa de entrega adicional, no la única verdad.

13.3 Modelo de entrega

- evento de dominio → outbox
- worker crea notificación persistente
- emite websocket si el usuario está conectado

13.4 Beneficio

Si falla websocket o el usuario está offline, la notificación sigue existiendo en backend.

---

14. Seguridad

14.1 Autenticación

- JWT access corto
- refresh token revocable
- hash seguro de contraseña
- rotación de refresh recomendable

14.2 Autorización

La autorización debe combinar:

- identity del usuario
- ownership del recurso
- role operativo sobre la interacción
- estado actual del recurso

No alcanza con “estar logueado”.

14.3 Rate limiting

Aplicar al menos en:

- login
- register
- upload de fotos
- creación de interacciones
- envío de mensajes
- reportes

14.4 Validación de archivos

- mime type permitido
- size máximo
- cantidad máxima por listing
- sanitización de nombres
- generación propia de object keys

14.5 Sanitización de contenido

Mensajes y campos textuales deben pasar por límites claros:

- longitud máxima
- trimming
- eventual normalización unicode

14.6 Secret management

- variables de entorno estrictas

- no secretos hardcodeados
- separación por ambiente

---

15. Observabilidad y operación

15.1 Logging estructurado

Todo log importante debe incluir contexto:

- requestId / correlationId
- userId si aplica
- resourceId
- action
- result
- errorCode si falla

15.2 Health endpoints

Tener mínimo:

- liveness
- readiness
- versión / commit SHA

15.3 Métricas iniciales

- requests por endpoint
- latencia p95
- errores 4xx / 5xx
- jobs pendientes / fallidos
- auditorías aprobadas / observadas / rechazadas
- tiempo medio hasta aceptación o respuesta

15.4 Audit trail

No hace falta sobreconstruir un event store completo, pero sí registrar eventos operativos clave.

Ejemplos:

- listing state changed
- interaction accepted
- interaction rejected
- match completed
- reservation expired
- moderation resolved

16. Estrategia de testing

16.1 Pirámide recomendada

Unit tests

Para:

- domain services
- policies
- value objects
- transiciones de estado
- validaciones críticas

Integration tests

Para:

- repositories
- transacciones de aceptación
- queries de read models
- workers de outbox o jobs críticos

E2E

Para flujos P0:

- registro + bootstrap
- crear listing + subir fotos + submit review
- aprobación y publicación
- crear purchase intent
- aceptar interacción → listing reservado + match abierto
- conversación → confirm success → listing cerrado
- expiración de reserva → listing vuelve a published

16.2 Prioridad real

En MVP, los tests más valiosos son:

- transiciones de estado
- ownership
- concurrencia en aceptación
- cierres automáticos incompatibles
- expiración de reserva

17. Versionado y evolución

17.1 API

Versionar desde el inicio con `/v1`.

17.2 Catálogos enumerados

Mantener catálogos oficiales centralizados para:

- categorías
- subcategorías
- talles
- condiciones
- razones de moderación
- quick actions

17.3 Cambios compatibles

Agregar campos nuevos sin romper consumidores. Evitar cambios destructivos prematuros.

---

18. Recomendaciones de implementación en NestJS

18.1 Controllers delgados

Los controllers solo deberían:

- validar request DTO
- resolver usuario autenticado
- delegar a application services
- mapear response DTO

18.2 Services con responsabilidad clara

Separar:

- command services para mutaciones
- query services para read surfaces

18.3 Repositories explícitos

No dejar que cualquier service consulte cualquier tabla sin criterio. Los repositories deben seguir ownership razonable por módulo.

18.4 Mappers explícitos

Necesitás mappers entre:

- entidades ORM ↔ dominio
- dominio ↔ read DTOs

18.5 Filtros y errores

Implementar error handling unificado con:

- codes tipados
- status HTTP coherente
- envelope estándar

---

19. Roadmap técnico recomendado de arranque

Fase A — Bootstrap base

- NestJS app
- config module
- PostgreSQL
- TypeORM
- migrations
- auth base
- swagger
- health
- logging base
- error envelope global

Fase B — Profiles + Listings base

- users/profiles
- reach zones
- create listing draft
- upload photos
- submit review
- listing detail
- my listings

Fase C — Moderation pipeline mínima

- image audit jobs
- moderation review
- observed/rejected/approved
- publication flow completo

Fase D — Discovery

- swipe feed
- saved listings
- dismiss
- filtros iniciales

Fase E — Interactions

- purchase intents
- trade proposals
- incoming interactions
- aceptación / rechazo
- cierre automático incompatible

Fase F — Matches + Conversations

- match sessions
- conversation thread
- messages
- quick actions
- confirm success / mark failed / cancel

Fase G — Notifications + Reputation

- notifications persistidas
- websocket realtime
- reputation entry
- profile trust metrics

Fase H — Jobs automáticos y robustez

- expiración de reserva
- archive técnico de conversaciones
- outbox worker estable
- rate limiting
- idempotency keys
- observabilidad ampliada

---

20. Decisiones técnicas cerradas v1

Cerrado

- monolito modular con NestJS
- PostgreSQL como verdad transaccional

- TypeORM + migraciones
- REST JSON versionado
- WebSockets para realtime
- Redis + BullMQ para jobs
- storage S3-compatible para imágenes
- outbox pattern simple
- transacciones en mutaciones críticas
- surfaces backend-driven con `state`, `viewerContext` y `availableActions`

No hacer todavía

- microservicios
- event sourcing completo
- CQRS rígido ceremonial
- presigned uploads complejos desde día 1
- proyecciones materializadas para todo
- scoring avanzado de IA
- pagos integrados

---

21. Riesgos técnicos que esta arquitectura busca evitar

- lógica crítica escondida en frontend
- endpoints CRUD pobres que después obliguen a parches
- múltiples verdades de disponibilidad
- chats abiertos sin contexto transaccional
- condiciones de carrera al aceptar interacciones
- acoplamiento fuerte entre moderación y publicación
- pérdida de eventos por falta de outbox
- crecimiento caótico sin ownership claro por módulo

---

22. Resultado esperado de esta etapa

Con esta arquitectura backend v1, CirculAR queda listo para pasar a una fase de implementación real sin improvisación estructural.

La siguiente etapa recomendada es:

1. modelo relacional inicial
2. catálogo de enums y constantes oficiales
3. scaffold técnico del proyecto NestJS
4. roadmap de implementación por módulos con criterios de aceptación